4단계 주제 3/6

Azure OpenAI · Microsoft Foundry

코딩 · Azure AI 기초

K뉴딜 MS Copilot 활용 과정

Copilot부터 AI 에이전트 · 업무자동화

4단계 · 코딩 · Azure AI 기초

마치면 할 수 있어요

- [ ] Azure AI Foundry가 Microsoft Foundry로 이름이 바뀐 사정을 알고 옛 자료를 가려 읽을 수 있습니다.
- [ ] Foundry 포털에서 프로젝트를 만들고 모델을 배포할 수 있습니다.
- [ ] API 키를 코드 밖 환경변수에 두고 안전하게 다룰 수 있습니다.
- [ ] Python으로 배포한 모델을 불러 답을 받을 수 있습니다.

이름과 낱말부터 바로 알기

1. 검색하면 이름이 제각각이라 헷갈립니다.
2. 같은 제품의 옛 이름이 함께 떠돌기 때문입니다.
3. Microsoft Learn 기준으로 지금 이름은 Microsoft Foundry입니다.
4. 블로그나 영상에서 옛 이름이 보이면 예전 자료일 수 있으니 날짜를 확인해 보세요.

이름과 낱말부터 바로 알기

옛 이름	지금 이름
Azure AI Studio, Azure AI Foundry	Microsoft Foundry
Azure AI Services	Foundry Tools
Foundry 옛 포털	Foundry (classic). 포털 위쪽 토글로 새 화면과 오갈 수 있습니다
모델 서비스 Azure OpenAI	Azure OpenAI in Microsoft Foundry Models

이름과 낱말부터 바로 알기 (1/2)

낱말	뜻	비유
Foundry 리소스	Azure에 만드는 본체입니다	식당 건물
프로젝트	리소스 안에서 내 작업을 모으는 칸입니다	식당 안 우리 팀 주방
모델	글을 만들어 내는 AI 자체입니다	레시피를 아는 요리사
배포(Deployment)	모델을 내 리소스에서 쓸 수 있게 올린 것입니다	요리사를 우리 주방에 모셔 오기

이름과 낱말부터 바로 알기 (2/2)

낱말	뜻	비유
배포 이름	코드에서 부르는 이름입니다. 기본값은 모델 이름이고 바꿀 수 있습니다	주방에서 부르는 호칭
엔드포인트	요청을 보낼 인터넷 주소입니다	식당 주소
API 키	요청하는 사람이 맞는지 확인하는 비밀값입니다	주방 출입증

프로젝트 만들고 모델 배포하기 (1/3)

1. Windows · macOS 같습니다.
2. 브라우저에서 진행합니다.
3. ai.azure.com 에 로그인하고 New Foundry 토글이 켜져 있는지 봅니다.
4. 새 프로젝트 만들기(Create a new project)를 고르고 프로젝트 이름을 입력합니다.
5. Advanced options에서 Resource group을 rg-rag-study로 고르고 Location을 정합니다.

프로젝트 만들고 모델 배포하기 (2/3)

1. Create project를 누르고 개요 화면이 뜰 때까지 기다립니다.
2. Build로 들어가 Models를 엽니다.
3. Deploy에서 Deploy a base model을 고르고 모델을 찾습니다.
4. Deploy에서 Default settings를 고릅니다.
5. 배포 상태가 Succeeded인지 보고 배포 이름을 메모합니다.

프로젝트 만들고 모델 배포하기 (3/3)

1. 배포가 끝나면 플레이그라운드(Playground)에서 코드 없이 먼저 말을 걸어 보세요.
2. 여기서 답이 나오면 배포는 정상입니다.
3. 코드에서 오류가 나도 원인을 코드 쪽으로 좁힐 수 있습니다.

프로젝트 만들고 모델 배포하기

1 **참고: 화면 표기는 바뀔 수 있습니다. 같은 이름이 안 보이면 비슷한 낱말을 찾거나 공식 문서를 함께 열어 보세요. 공식 빠른 시작은 gpt-5-mini를 예로 쓰지만 모델 목록과 지역별 가용 여부, 할당량은...**

키와 주소를 코드 밖에 두기

1. 배포의 상세 화면에서 엔드포인트와 키를 볼 수 있습니다.
2. 이 값을 .env 파일에 옮겨 적으면 코드에는 키가 한 글자도 남지 않습니다.
3. 주의: API 키는 현관 비밀번호와 같습니다. 코드에 적거나 깃허브에 올리거나 화면 공유, 캡처, 질문 게시글에 노출하면 안 됩니다. 남이 쓴 만큼 내 구독에 요금이 청구될 수 있습니다. 깃허브에 올렸다면 커밋을...

키와 주소를 코드 밖에 두기

1. Windows · macOS 같습니다.
2. VS Code에서 프로젝트 폴더에 파일 두 개를 만들어 보세요.
3. 파일 이름 앞의 점까지 그대로 씁니다.

키와 주소를 코드 밖에 두기

```
# .env : 비밀값을 모아 두는 파일입니다. 깃허브에 절대 올리지 않습니다.
# 값 양옆에 따옴표와 공백 없이 씁니다.
AZURE_OPENAI_API_KEY=여기에_내_키_붙여넣기
AZURE_OPENAI_BASE_URL=https://내리소스이름.openai.azure.com/openai/v1/
AZURE_OPENAI_DEPLOYMENT=내가_정한_배포_이름
```

키와 주소를 코드 밖에 두기

```
# .gitignore : 깃허브에 올리지 않을 파일 목록입니다.
.env
.venv/
__pycache__/
```

키와 주소를 코드 밖에 두기

1 **참고: base_url에는 /openai/v1/로 끝나는 주소를 넣습니다. 프로젝트 첫 화면에 보이는 .../api/projects/... 주소는 Foundry SDK용이라 이 코드에 그대로 넣지 않습니다.**

키와 주소를 코드 밖에 두기 · Windows

1. .env 없이 터미널에 값을 직접 넣어도 코드는 똑같이 돌아갑니다.
2. 터미널을 닫으면 값이 사라집니다.

키와 주소를 코드 밖에 두기 · Windows

```
$env:AZURE_OPENAI_API_KEY = "여기에_내_키_붙여넣기"
```

> 주석을 포함한 전체 코드는 아래 본문에 있습니다.

키와 주소를 코드 밖에 두기 · macOS

1. .env 없이 터미널에 값을 직접 넣어도 코드는 똑같이 돌아갑니다.
2. 터미널을 닫으면 값이 사라집니다.

키와 주소를 코드 밖에 두기 · macOS

```
export AZURE_OPENAI_API_KEY="여기에_내_키_붙여넣기"
```

> 주석을 포함한 전체 코드는 아래 본문에 있습니다.

Python으로 첫 호출 보내기

1. Windows · macOS 같습니다.

2. 가상환경이 켜진 터미널에서 설치하고 아래 코드를 first_call.py로 저장한 뒤 실행해 보세요.

Python으로 첫 호출 보내기

```
pip install openai python-dotenv
python first_call.py
```

Python으로 첫 호출 보내기 (1/2)

```python
import os

from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.environ["AZURE_OPENAI_API_KEY"],
    base_url=os.environ["AZURE_OPENAI_BASE_URL"],
)

response = client.responses.create(
    model=os.environ["AZURE_OPENAI_DEPLOYMENT"],
    instructions="너는 친절한 학습 도우미야. 중학생도 이해하게 세 문장으로 답해줘.",
    input="클라우드가 뭔지 비유로 설명해줘.",
```

> **주석을 포함한 전체 코드는 아래 본문에 있습니다.**

Python으로 첫 호출 보내기 (2/2)

```
)

print(response.output_text)
```

주석을 포함한 전체 코드는 아래 본문에 있습니다.

Python으로 첫 호출 보내기

인터넷에서 보이는 코드	지금 공식 문서의 v1 방식	왜 달라졌나요
AzureOpenAI() 클라이언트	OpenAI() 클라이언트	OpenAI와 같은 클라이언트를 그대로 쓰게 되었습니다
azure_endpoint=...	base_url="…/openai/v1/"	주소 끝에 /openai/v1/을 붙입니다
api_version="2024-..."	쓰지 않습니다	v1 API에서는 날짜별 버전 값이 필수가 아닙니다
model="모델 이름"	model="내 배포 이름"	Azure에서는 배포 이름으로 찾아갑니다

Python으로 첫 호출 보내기

증상	먼저 볼 곳
KeyError: 'AZURE_OPENAI_API_KEY'	.env 파일 이름과 위치, 낱말 철자
401 인증 오류	키 복사가 잘렸는지, 앞뒤 공백이 붙었는지
404 오류	배포 이름 철자, base_url이 /openai/v1/로 끝나는지
429 요청 초과	잠시 기다린 뒤 다시 실행하고 배포의 할당량을 봅니다
ModuleNotFoundError	가상환경이 켜졌는지, pip install을 했는지

4단계 · 코딩 · Azure AI 기초

실습

시연을 보고, 같은 결과를 따라 만든 뒤, 내 주제로 바꿉니다.

실습 1

시연을 봅니다

실습

1 새 프로젝트에 모델을 배포하고 플레이그라운드에서 답을 본 뒤 같은 질문을 Python 코드로 보내는 과정을 함께 봅니다.

실습 2

따라 합니다 (1/2)

실습

1. Foundry 포털에서 rg-rag-study 안에 프로젝트를 만듭니다.
2. 모델 하나를 배포하고 배포 이름을 메모합니다.
3. 플레이그라운드에서 질문을 보내 답을 확인합니다.
4. .env와 .gitignore 파일을 만듭니다.

실습 2

따라 합니다 (2/2)

실습

5 엔드포인트와 키, 배포 이름을 .env에 넣습니다.

6 pip로 openai와 python-dotenv를 설치합니다.

7 first_call.py를 저장하고 실행합니다.

8 input 문구를 바꿔 다시 실행합니다.

실습 2

복사해서 쓰는 프롬프트

```
내 프로젝트의 first_call.py를 읽고 줄마다 무엇을 하는지 중학생도 알게 설명해줘. 그다음 instr
    uctions를 [내가 원하는 역할, 예: 면접 코치]로 바꾸고 input을 터미널에서 입력받도록 코드
    를 고쳐줘. API 키는 코드에 쓰지 말고 지금처럼 환경변수에서만 읽어줘. 키 값을 나에게 묻지
    도 말아줘.
```

실습 2

체크포인트

시점	이렇게 되면 통과
10분	Foundry 포털에 내 프로젝트의 개요 화면이 보입니다.
20분	배포 목록에서 내 배포 상태가 Succeeded이고 플레이그라운드가 답합니다.
30분	.env와 .gitignore가 있고 코드에는 키가 없습니다.
40분	python first_call.py를 실행하면 터미널에 모델의 답이 나옵니다.

실습 3

내 주제로 바꿉니다

실습

1. instructions와 input을 내 관심사로 바꿔 보세요.
2. 면접 예상 질문을 내 주는 도우미처럼 역할을 바꾸면 같은 코드가 다른 도구가 됩니다.

과제

기본 과제와 도전 과제

기본	내가 정한 역할과 질문으로 first_call.py를 실행해 답을 받아 보세요. 결과를 캡처할 때 키가 화면에 없는지 확인합니다.
도전	공식 빠른 시작을 읽고 키 대신 az login과 DefaultAzureCredential로 로그인하는 방식을 시도해 보세요: learn.microsoft.com

코딩 · Azure AI 기초 마무리

Azure OpenAI · Microsoft Foundry

포트폴리오에 넣을 것: 환경변수로 키를 읽어 모델을 호출하는 first_call.py와 실행 결과 캡처

- ✓ **Azure AI Foundry가 Microsoft Foundry로 이름이 바뀐 사정을 알고 옛 자료를…**
- ✓ **Foundry 포털에서 프로젝트를 만들고 모델을 배포할 수 있습니다.**
- ✓ **API 키를 코드 밖 환경변수에 두고 안전하게 다룰 수 있습니다.**
- ✓ **Python으로 배포한 모델을 불러 답을 받을 수 있습니다.**